

Лабораторна робота №2.

Тема: Операції над цілими числами, форматований вивід

Мета: Опанувати навички виконання арифметичних та цілочисельних операцій над цілими числами (`int`) у мові Python; навчитися розв'язувати алгоритмічні задачі на роботу з розрядами чисел, а також опанувати сучасні засоби форматowanego виводу даних (`f-strings`).

КОРОТКІ ТЕОРЕТИЧНІ ВІДОМОСТІ

1. Арифметичні та цілочисельні операції

У мові Python для цілих чисел (`int`) окрім стандартних операцій додавання (+), віднімання (-) і множення (*) використовуються спеціальні:

- `//` – цілочисельне ділення (відкидає дробову частину, повертаючи ціле число).
- `%` – залишок від цілочисельного ділення (наприклад, `17 % 5` дорівнює 2).
- `**` – піднесення до степеня.

Операції `//` та `%` є базовими для виділення окремих розрядів числа (одиниць, десятків, сотень тощо).

2. Форматований вивід (`f-strings`)

Для красивого оформлення результатів розрахунків використовують інтерполяцію рядків (`f-strings`). У середині фігурних дужок `{}` можна вказувати не лише змінні, а й специфікатори форматування:

- Задання ширини поля та вирівнювання: `{value:>10}`.
- Вивід цілих чисел із лісуючими нулями (наприклад, для годин чи днів): `{day:02d}`.
- Вивід результатів обчислень та округлень безпосередньо у тексті.

Ось приклади з наведеного зображення, переписані у сучасному текстовому форматі з використанням **`f-strings`** (форматованих рядків), які є більш зручним та стандартним інструментом у Python порівняно з методом `.format()`.

Для демонстрації візьмемо змінні:

Python

```
n = 25
```

```
f = 9.03
```

```
s = "search string"
```

1. Мінімальна довжина поля становить 10 для кожного значення, вирівнювання по правому краю (за замовчуванням для чисел) або за лівим (для рядків):

Python

```
| print(f"{n:10d}{f:10f}{s:10s}")
```

2. Мінімальна довжина поля становить 20 для кожного значення, вирівнювання по правому краю (>):

Python

```
| print(f"{n:>20d}{f:>20f}{s:>20s}")
```

3. Мінімальна довжина поля становить 15 для кожного значення, вирівнювання по лівому краю (<):

Python

```
| print(f"{n:<15d}{f:<15f}{s:<15s}")
```

4. Мінімальна довжина поля становить 10 для кожного значення, вирівнювання по центру (^):

Python

```
| print(f"{n:^10d}{f:^10f}{s:^10s}")
```

Приклад 1.

Вводиться цифра 0 або 1, необхідно вивести 1 або 0 відповідно, не використовуючи конструкцію розгалуження.

Для розв'язання цієї задачі без використання конструкцій розгалуження (if-else чи подібних) можна скористатися простою математичною операцією інверсії.

Оскільки вводиться лише одне з двох чисел – 0 або 1, протилежне значення можна отримати за допомогою віднімання введенного числа від одиниці:

$$\text{result} = 1 - x$$

Приклад програмного коду на Python:

Python

```
# Ввід числа (0 або 1)
```

```
x = int(input("Введіть 0 або 1: "))
```

```
# Обчислення інвертованого значення без використання умовних операторів
```

```
result = 1 - x
```

```
# Вивід результату
```

```
print(f"Результат: {result}")
```

Як це працює:

- Якщо користувач введе **0**, то програма обчислить $1 - 0$, що дорівнює **1**.
- Якщо користувач введе **1**, то програма обчислить $1 - 1$, що дорівнює **0**.

Таким чином, відбувається повна заміна цифри на протилежну суто через арифметичні дії.

Приклад 2.

Автомобіль може проїхати відстань n кілометрів за день. Скільки днів пройде для проїзду маршруту довжиною m кілометрів? Програма отримує два цілих додатних числа: n і m .

Для розв'язання цієї задачі потрібно визначити кількість днів, необхідну для подолання маршруту довжиною m кілометрів, якщо за день автомобіль проїжджає n кілометрів.

Оскільки маршрут може не ділитися націло на денну норму пробігу, а шлях потрібно подолати повністю (тобто навіть якщо залишився маленький шматочок шляху, потрібен ще один додатковий день), **тут застосовується математичне округлення вгору**. Це можна зробити за допомогою цілочисельного ділення за формулою:

$$\text{days} = (m + n - 1) // n$$

Python

```
print("--- Розрахунок тривалості поїздки ---")

# Крок 1: Ввід добового пробігу (n) та загальної довжини маршруту (m)
n = int(input("Введіть кілометрів за день (n): "))
m = int(input("Введіть загальну довжину маршруту в км (m): "))

# Крок 2: Обчислення кількості днів з округленням вгору без умовних операторів
days = (m + n - 1) // n

# Крок 3: Форматований вивід результатів з використанням f-strings та вирівнювання
print("\n" + "=" * 35)
print(f"{'ЗВІТ МАРШРУТУ':^35}")
print("=" * 35)
print(f"{'Денна норма (n):':<20} {n:>12d} км")
print(f"{'Довжина шляху (m):':<20} {m:>12d} км")
print("-" * 35)
print(f"{'Потрібно днів:':<20} {days:>12d}")
print("=" * 35)
```

Пояснення формули:

Вираз $(m + n - 1) // n$ дозволяє заокруглити результат ділення m на n у більшу сторону без використання функцій з модуля `math` чи умовних операторів (`if-else`). Якщо m кратне n , додавання $(n - 1)$ корегує залишок так, щоб отримати точну кількість днів без зайвого заокруглення.

ЗАВДАННЯ ДЛЯ САМОСТІЙНОЇ РОБОТИ

Напишіть програми мовою Python, забезпечивши зручний ввід даних та акуратний форматований вивід результатів.

Рівень I. Розряди, залишки та форматування.

Завдання 1.1. Аналіз двоцифрового числа.

Користувач вводить ціле двоцифрове число. Програма повинна за допомогою операторів `//` та `%` виділити десятки й одиниці, обчислити їх суму та вивести детальний звіт у вигляді гарно оформленого рядка (`f-string`) із вирівнюванням.

Завдання 1.2. Конвертер часу: секунди у хвилини.

Користувач вводить загальну кількість секунд (ціле число). Переведіть їх у хвилини та залишкові секунди. Виведіть результат у строго форматovanому вигляді, наприклад: «Час: 02 хв : 15 сек» (використовуйте специфікатор із провідними нулями: `%02d`).

Завдання 1.3. Обчислення вартості книг.

Книга коштує a гривень і b копійок. Визначте, скільки гривень і копійок потрібно заплатити за n книг. Значення вводяться користувачем у порядку a , b , n на окремих рядках, а сума до сплати в одному рядку через пропуск: кількість гривень і копійок відповідно.

- Вхідні дані: 10 15 2
- Вихідні дані: 20 30.

Рівень II. Трицифрові числа та математичні алгоритми.

Завдання 2.1. Дзеркальний розворот трицифрового числа.

Користувач вводить ціле трицифрове число. Використовуючи виключно цілочисельну арифметику (`//` та `%`), розбийте число на розряди, сформууйте нове число із перевернутим порядком цифр та виведіть результати у вигляді табличного звіту з чітким вирівнюванням по стовпцях.

Завдання 2.2. Заокруглення до найближчого більшого кратного без умов

Дано два цілі числа: a та b ($b > 0$). За допомогою формули на основі цілочисельного ділення та залишку знайдіть найменше число, кратне b , яке є більшим або рівним за a (без використання умовних операторів `if` чи

вбудованих функцій округлення). Виведіть вхідні дані та результат за допомогою `f-string` із фіксованою шириною полів.

Завдання 2.3. Аналіз залишків від ділення.

Користувач вводить три цілі числа. За допомогою арифметичних операцій визначте, чи ділиться перше число на друге без залишку, і виведіть результат перевірки у вигляді форматowanego логічного звіту.